

HW#1 데이터통신 프로그래밍 과제

분산 환경에서의 Fault-Tolerant 키-값 저장소 구현

(Multi-Thread 소켓 통신 · 동적 부하 분산 · 장애 복구)

○ DUE:	2026.9.23(수) 23:59 (Assigned: 2026.9.10(목) 09:00)
○ How:	1개 파일 제출 – G조이름HW1.zip (조별로 한명만 제출) ex) G1HW1.zip
	모든 소스파일 및 결과파일 전체를 압축한 G조이름HW1.zip으로 제출

0. 분산 키-값 저장소 기본 개념

분산 키-값 저장소(Distributed Key-Value Store)는 대규모 데이터를 여러 노드에 분산 저장하고 고가용성을 보장하는 저장 시스템이다. Master-Worker 구조에서 Master는 데이터 분배와 장애 처리를 조율하고, Worker는 실제 데이터를 저장·처리한다.

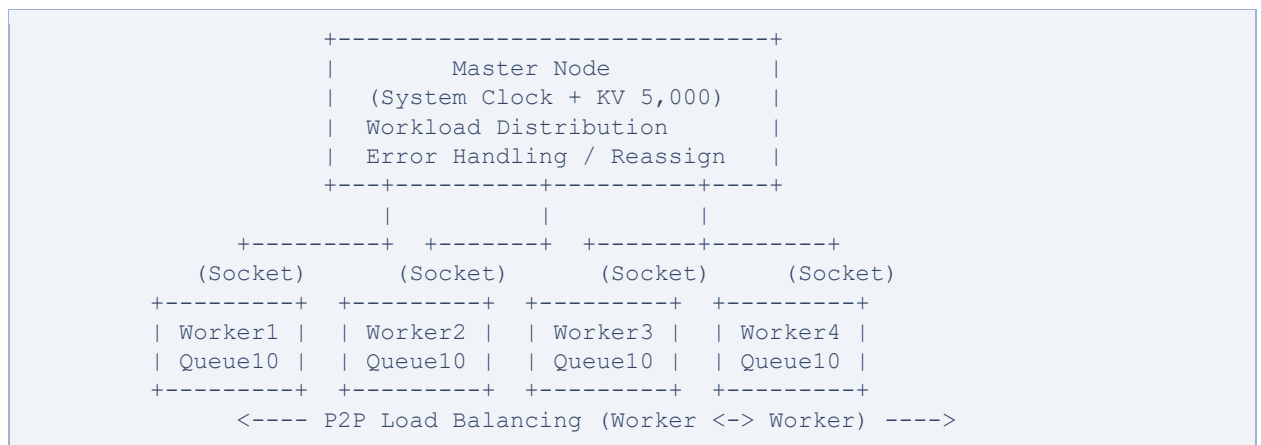
0-1. 핵심 개념

개념	설명
Key-Value Store	Key는 고유 식별자(ID), Value는 저장할 데이터(1~100 정수). Map 자료구조 기반으로 O(1) 접근을 제공한다. (Key는 Unique한 16진수 4자리수 예:a3f7)
Master Node	System Clock 관리, 5,000개 KV 쌍 생성, Worker에 작업 동적 분배, 장애 감지 및 재할당 수행.
Worker Node	Ready Queue(최대 10개, 70% Full 초과 Warning, 10개초과 FAIL)로 작업 수신, 1~3sec 랜덤 처리시간, 80% 성공률, P2P 부하 분산 참여.
Ready Queue	워커 노드의 대기 작업 버퍼. 큐 크기는 최대 10개이며, 초과 요청은 즉시 FAIL 처리된다. WARN은 상태 전이 시 1회가 아니라, 큐가 70%를 초과한 상태에서 작업이 들고날 때마다 매번 기록한다. (70% Full 초과 Warning, 10개초과 FAIL)
System Clock	실제 시간이 아닌 가상의 연산·네트워크 지연 누적 시간. 시스템클럭 (단위 sec) 유지 및 관리. 모든 이벤트 로그에 함께 기록된다. ※ Worker 처리시간 (1~3초)과 노드간 통신 지연(1초)은 실제 Thread sleep이 아니라 System Clock 값에 더하는 가상 누적치이다. 프로그램 자체는 실시간 대기 없이 최

	대한 빠르게 실행된다.
Fault Tolerance	Worker 실패(20%) 시 Master가 감지하여 해당 작업을 다른 Worker에 재할당, 데이터 유실을 방지한다. 재할당된 작업에도 동일한 80%/20% 규칙이 다시 적용되며, 또 실패하면 성공할 때까지 최우선 큐에 재등록해 재시도한다.

0-2. 시스템 구조 및 데이터 흐름

[Master-Worker 구조도]



0-3. 동적 작업 분배 알고리즘 예시

Master는 각 Worker의 큐 상태를 모니터링하여 작업을 동적으로 분배한다. 아래는 분배 판단 예시이다.

시점	Worker1 Queue	Worker2 Queue	Worker3 Queue	Master 판단
t=1s	3 / 10	2 / 10	1 / 10	Worker3에 다음 작업 할당 (최소 큐)
t=5s	8 / 10	5 / 10	2 / 10	Worker3 우선 할당 / Worker1 포화 주의
t=9s	10 / 10	9 / 10	4 / 10	Worker1 큐 초과 → 해당 요청 FAIL 처리
t=12s	FAIL	6 / 10	3 / 10	Worker1 장애 → 작업 Worker3에 재할당

0-4. P2P 부하 분산 동작 예시

Worker 노드끼리 직접 소켓 통신으로 부하를 분산한다. 큐 대기 시간이 임계값(15sec)을 초과하면 인접 Worker에게 작업 이전을 요청한다. 예상 대기시간 = 현재 큐에 남은 작업 수 × 평균 처리시간(2초)으로 계산한다. 각 Worker는 1~3초 사이의 랜덤한 주기로 이 값을 반복 점검한다.

단계	주체	동작
1	Worker1	자신의 큐 대기 시간 모니터링: 현재 큐=9개, 평균 처리=2s → 예상 대기=18s (임계 초과)
2	Worker1	인접 Worker2, Worker3 큐 상태 조회. Worker3 큐=3개 확인.
3	Worker1 -> Worker3	큐 하위 3개 작업을 Worker3으로 이전 요청 (P2P 소켓 전송).
4	Worker3	작업 수신 후 자신의 큐에 추가. 수신 확인(ACK) 전송.
5	Worker1	ACK 수신 후 이전된 작업 큐에서 제거. 균형 복원 완료.

0-5. 장애 처리(Fault Tolerance) 흐름 예시

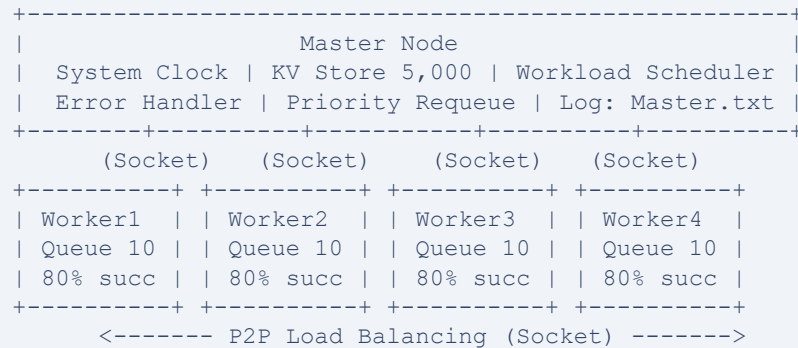
Worker2가 작업 처리 중 실패(20% 확률)를 Master에 보고하는 경우:

- Worker2가 작업(KV 쌍 저장)을 처리하던 중 실패 발생 → Master에 FAIL 메시지 전송.
- Master는 실패한 작업 ID를 최우선 큐(Priority Queue)에 등록한다.
- Master는 현재 큐가 가장 여유 있는 Worker(예: Worker4)를 선택하여 재할당.
- 재할당된 Worker4가 동일 작업을 80%/20% 규칙으로 재처리한다(재할당 작업도 실패 확률이 동일하게 적용됨). SUCCESS 시 결과를 Master에 전송하고, FAIL 시 다시 최우선 큐에 등록되어 다른 Worker에 재재할당된다.
- Master는 KV 저장소/결과 기록을 업데이트하고 전체 진행률을 갱신한다.

1. 시뮬레이션 환경 및 구성요소

1개의 Master Node와 4개의 Worker Node로 구성된 분산 키-값 저장소 시스템을 구현한다. 모든 노드 간 통신은 Socket(TCP)으로 구현하며, 각 Worker Node는 독립적인 Thread로 동작한다. Master Node는 외부 클라우드 서버(AWS/GCP)에 반드시 구현되어야 한다. Worker Node(4개 Thread)는 조원의 로컬 PC에서 실행하며, Master의 공인(public) IP와 포트로 접속한다. Master의 IP/포트는 프로그램 실행 시 인자(argument) 또는 설정파일로 지정할 수 있도록 구현한다.

[시스템 구성도]



① Master Node (Controller)

- 외부 서버 구축: **Master Node** 는 반드시 **AWS, Google Cloud** 등 물리적 외부 서버에 구현되어야 한다.
- 데이터 생성: 프로그램 시작 시 **5,000** 개의 (**Key, Value**) 쌍을 생성한다. **Key** 는 고유 ID, **Value** 는 1~100 의 무작위 정수.
- System Clock** 관리: 전역 가상 시계를 유지하며, 모든 이벤트 발생 시 해당 시각을 로그에 기록한다. **Worker** 의 처리·통신 지연 값도 이 가상 시계에 누적될 뿐, 프로그램의 실제 실행 속도(**real time**)와는 무관하다.
- 동적 작업 분배: **Worker Node** 의 현재 처리량과 **Queue** 상태를 모니터링하여 데이터를 동적으로 분배한다.
- 장애 처리(**Fault Tolerance**): **Worker** 로부터 처리 실패 보고를 받으면 해당 작업을 **Priority Queue** 에 등록하여 다른 **Worker** 에 최우선 재할당한다.
- 최종 출력: 전체 처리 완료 시 **KV** 저장소 전체(**5,000** 쌍), 총 수행시간, 노드별 처리 통계를 로그에 기록한다.
- 모든 이벤트를 **Master.txt** 로그에 기록하며, 처리 완료 후 **Graceful Termination** 을 수행한다.

② Worker Node (Storage & Processor, 4 개)

각 **Worker Node**는 독립 **Thread**로 동작하며 아래 기능을 수행한다.

- Ready Queue** 운영: 최대 **10** 개의 작업을 대기시킬 수 있으며, 초과 요청은 즉시 **FAIL** 로 처리된다.
- 가상 처리 시간: 각 작업당 **1~3** 초의 무작위 처리 시간이 소요된다고 가정한다. 실제로 스레드가 그 시간만큼 대기(**sleep**)하는 것이 아니라, **System Clock** 값에 해당 시간을 더하는 방식으로 시뮬레이션하며 프로그램 실행 자체는 지연 없이 빠르게 진행된다.
- 성공 확률 시뮬레이션: 모든 작업은 **80%** 성공 / **20%** 실패 확률을 가진다. 실패 시 **Master** 에 **FAIL** 메시지를 전송한다.
- P2P** 부하 분산: 자신의 큐 대기 시간이 임계값(**15sec**)을 초과하면 인접 **Worker** 와 소켓 통신으로 작업을 이전한다. 예상 대기시간은 (큐에 남은 작업 수 × 평균 처리시간 **2** 초)로 계산하며, **1~3** 초 랜덤 주기로 반복 점검한다.

- 통신 지연: 모든 노드 간 네트워크 지연은 1 초로 고정하여 시뮬레이션한다. 이 역시 실제 대기가 아닌 **System Clock** 누적 값으로만 반영한다.
- 최종 출력: 연결 종료 전 성공/실패 횟수, 작업 처리량, 평균 대기시간, 전체 수행시간을 출력한다.
- 각 Worker 간 모든 이벤트를 Worker1.txt ~ Worker4.txt 로그에 기록한다.

※ 필수 구현 조건 (하나라도 미충족 시 과제 0점 처리)

- **Master Node** 는 반드시 AWS, GCP 등 물리적 외부 서버에 구현되어야 함
- **Master Node** 와 모든 **Worker Node** 간 통신은 반드시 **Socket** 으로 구현되어야 함
- 모든 **Worker Node** 는 독립적인 **Thread** 를 통해 구현되어야 함

2. 시뮬레이션 시나리오

다음 시나리오에 따라 시스템을 동작시키고 각 노드별 로그를 기록한다. Master와 4개의 Worker 노드는 각각 Master.txt, Worker1.txt ~ Worker4.txt에 모든 이벤트와 결과를 기록한다.

Step 1 - 초기화 (System Clock: 0 sec)

- Master Node 가 먼저 구동되어 System Clock 을 0 에서 시작한다.
- Master 가 5,000 개의 (Key, Value) 쌍을 생성하고 메모리에 저장한다.
- 4 개의 Worker Node 가 순차적으로 소켓 연결을 완료하고 Ready Queue 를 초기화한다.
- Master 가 전체 초기화 완료 후 작업 분배를 시작한다.

Step 2 - 동적 작업 분배 (0 sec ~)

- Master 는 각 Worker 의 현재 Queue 상태를 지속 모니터링한다.
- Queue 여유 공간이 가장 많은 Worker 에게 우선적으로 작업을 분배한다.
- Worker Queue 가 최대(10 개)에 도달하면 해당 Worker 에 대한 분배를 일시 중단한다.
- 전체 5,000 개 작업을 분배 완료할 때까지 반복하며, 통신 지연은 1sec 로 가정한다.

Step 3 - 작업 처리 및 결과 수집

- 각 Worker 는 Queue 에서 작업을 꺼내 1~3sec 의 가상 처리 시간 후 결과를 Master 에 전송한다.
- 성공(80%) 시 Master 의 KV 저장소가 업데이트된다.
- 실패(20%) 시 Master 는 해당 KV 쌍을 Priority Queue 에 등록 후 가장 여유 있는 Worker 에 재할당한다.
- 재할당 시 기존 분배와 동일한 방식으로 공평하게 이루어지며, 재할당 작업이 최우선 처리된다. 재할당된 작업도 80% 성공 / 20% 실패 규칙이 동일하게 적용되고, 실패할 경우 다시 Priority Queue 에 등록되어 성공할 때까지 반복 재시도된다.

Step 4 - P2P 부하 분산 (상시 동작)

- 각 Worker 는 자신의 큐 대기 시간을 주기적으로 계산한다.

- 대기 시간이 임계값(15sec)을 초과하면 인접 Worker에 큐 상태를 조회하고 작업 이전을 요청한다.
- 부하 분산 알고리즘(방법론)은 제약 없으며, 제출 시 사용된 알고리즘을 반드시 명시한다.
- 부하 분산으로 이전된 모든 작업도 Worker.txt 로그에 기록한다.

Step 5 - 종료

- 전체 5,000 개 KV 쌍 처리가 완료되면 Master가 완료 신호를 모든 Worker에 전송한다.
- 각 Worker는 성공/실패 횟수, 작업 처리량, 평균 대기시간, 전체 수행시간을 출력 후 종료한다.
- Master는 KV 저장소 전체, 총 수행시간, 노드별 통계를 로그에 저장 후 Graceful Termination 한다.

3. 성능 평가 지표

다음 표에 정의된 6개 지표는 필수 측정 항목이며, 각 노드의 최종 통계 로그(STAT)에 반드시 포함해 기록한다. 4장의 로그 예시에 등장하는 추가 항목(Queue overflow rejects, Retry tasks received, P2P 전송/수신 세부 건수 등)은 참고용 예시이며 필수는 아니다.

지표명	단위	설명
Worker별 작업 처리량	건	각 Worker가 성공적으로 처리한 KV 쌍의 수
Worker별 성공/실패 횟수	건	전체 처리 중 성공 및 20% 실패 케이스 카운트
작업 평균 대기시간	sec	Queue에 들어온 후 처리 시작까지의 평균 대기 시간
P2P 부하 분산 이벤트 횟수	회	Worker 간 작업 이전이 발생한 총 횟수
장애 재할당 횟수	건	실패로 인해 다른 Worker에 재할당된 작업 수
전체 수행시간	sec	System Clock 기준 전체 시나리오 소요 시간

4. 노드별 Log 출력 예시

각 노드는 모든 이벤트를 System Clock과 함께 아래 형식으로 기록한다. STATUS 필드는 **INFO** / **SUCCESS** / **FAIL** / **WARN** 중 하나만 사용한다.

EVENT 필드는 고정된 목록이 아니라 자유 형식이다(예: INIT, CONNECT, DISTRIB, RESULT, RECV, PROC, LB, QUEUE, STAT, TERMINATE 등). 프로그램에서 정의한 모든 EVENT 종류는 제출물 AllDefinedLogs.txt에 명세·설명해야 한다.

공통 로그 형식: **[clock] NODE | EVENT | STATUS | message**

※ 아래 4-1~4-5의 시각(예: 195.40초), 진행률, 통계 수치는 모두 예시 값이다. 실제 실행 결과(수행시간, 성공/실패 건수 등)는 실행할 때마다 달라진다.

4-1. Master.txt 예시

```
=== Master.txt (Master Node Log) ===

=====
Master Node | Distributed KV Store | 2026-04-15
=====

[0.00] MASTER | INIT | INFO | System Clock started.
[0.00] MASTER | INIT | INFO | Generating 5,000 KV pairs... Key=hex4(unique),
Value=rand(1~100)
[0.02] MASTER | INIT | SUCCESS | 5,000 KV pairs generated. Waiting for Worker
connections.

[0.51] MASTER | CONNECT | SUCCESS | Worker1 connected. Ready Queue initialized
(0/10).
[0.52] MASTER | CONNECT | SUCCESS | Worker2 connected. Ready Queue initialized
(0/10).
[0.53] MASTER | CONNECT | SUCCESS | Worker3 connected. Ready Queue initialized
(0/10).
[0.54] MASTER | CONNECT | SUCCESS | Worker4 connected. Ready Queue initialized
(0/10).
[0.55] MASTER | INIT | INFO | All 4 Workers connected. Starting workload
distribution.

[1.00] MASTER | DISTRIB | INFO | Dispatching KV[0001] -> Worker3 (queue=1/10,
lowest)
[1.01] MASTER | DISTRIB | INFO | Dispatching KV[0002] -> Worker1 (queue=1/10)
[1.02] MASTER | DISTRIB | INFO | Dispatching KV[0003] -> Worker2 (queue=1/10)
[1.03] MASTER | DISTRIB | INFO | Dispatching KV[0004] -> Worker4 (queue=1/10)
[1.04] MASTER | DISTRIB | INFO | Dispatching KV[0005] -> Worker3 (queue=2/10)

[3.20] MASTER | RESULT | SUCCESS | KV[0001] stored by Worker3. value=47,
time=2.20s
[3.50] MASTER | RESULT | SUCCESS | KV[0002] stored by Worker1. value=83,
time=2.49s
[4.10] MASTER | RESULT | FAIL | KV[0003] FAILED by Worker2 (20% rule).
Requeuing...
[4.11] MASTER | DISTRIB | WARN | KV[0003] -> Priority requeue. Reassigning to
Worker4 (queue=3/10)
[4.12] MASTER | DISTRIB | SUCCESS | KV[0003] reassigned to Worker4.

[9.00] MASTER | DISTRIB | WARN | Worker1 queue full (10/10). Pausing dispatch
to Worker1.
[9.01] MASTER | DISTRIB | INFO | Redirecting to Worker3 (queue=4/10).
```

```

[15.00] MASTER | DISTRIB | INFO | Progress: 320 / 5000 KV pairs stored (6.4%)
[15.00] MASTER | DISTRIB | INFO | Worker1: queue=7, Worker2: queue=5, Worker3:
queue=3, Worker4: queue=6

[30.00] MASTER | DISTRIB | INFO | Progress: 710 / 5000 KV pairs stored (14.2%)

[120.00] MASTER | DISTRIB | INFO | Progress: 3100 / 5000 KV pairs stored (62.0%)

[195.40] MASTER | DISTRIB | SUCCESS | All 5,000 KV pairs distributed and
processed.
[195.40] MASTER | STAT | INFO | === FINAL STATISTICS ===
[195.40] MASTER | STAT | INFO | Total KV pairs processed : 5,000
[195.40] MASTER | STAT | INFO | Total SUCCESS : 3,987 (79.7%)
[195.40] MASTER | STAT | INFO | Total FAIL (then retry) : 1,013 (20.3%)
[195.40] MASTER | STAT | INFO | Total reassignments : 1,013
[195.40] MASTER | STAT | INFO | P2P load balance events : 34
[195.40] MASTER | STAT | INFO | Total execution time : 195.40 sec
[195.40] MASTER | STAT | INFO | --- Per-Worker Stats ---
[195.40] MASTER | STAT | INFO | Worker1: processed=1284, success=1023,
fail=261
[195.40] MASTER | STAT | INFO | Worker2: processed=1198, success=954,
fail=244
[195.40] MASTER | STAT | INFO | Worker3: processed=1312, success=1048,
fail=264
[195.40] MASTER | STAT | INFO | Worker4: processed=1206, success=962,
fail=244
[195.45] MASTER | TERMINATE | SUCCESS | Graceful shutdown. All workers
disconnected.
=====

```

4-2. Worker1.txt 예시 (정상 동작 + P2P 부하 분산)

```

=== Worker1.txt (Worker Node 1 Log) ===

```

```

Worker1 | Thread-based Worker | Ready Queue max=10

```

```

[0.00] WORKER1 | INIT | INFO | Worker1 thread started. Connecting to
Master...
[0.51] WORKER1 | CONNECT | SUCCESS | Connected to Master. Ready Queue initialized
(0/10).

[1.01] WORKER1 | RECV | INFO | Received task: KV[0002] (Key=a3f1, Value=83)
[1.01] WORKER1 | PROC | INFO | Processing KV[0002]... estimated time=2.49s
[3.50] WORKER1 | PROC | SUCCESS | KV[0002] stored. time=2.49s. Sending result
to Master.

[1.55] WORKER1 | RECV | INFO | Received task: KV[0009] (Key=b7c2, Value=21)
[1.56] WORKER1 | RECV | INFO | Received task: KV[0015] (Key=d1e4, Value=66)
[3.80] WORKER1 | PROC | SUCCESS | KV[0009] stored. time=2.25s
[4.20] WORKER1 | PROC | FAIL | KV[0015] FAILED (20% rule). Sending FAIL to
Master.
[4.21] WORKER1 | PROC | INFO | Master notified of failure for KV[0015].

[8.00] WORKER1 | QUEUE | WARN | Queue nearing full: 9/10. Checking
neighbors...
[8.01] WORKER1 | LB | INFO | Querying Worker2 queue: 4/10. Querying

```

```

Worker3: 2/10.
[8.02] WORKER1 | LB      | INFO    | Transferring 3 tasks to Worker3 (P2P socket).
[8.03] WORKER1 | LB      | INFO    | Sending KV[0088], KV[0089], KV[0090] ->
Worker3
[9.05] WORKER1 | LB      | SUCCESS | ACK received from Worker3. Queue updated:
6/10 -> 3/10.

[9.00] WORKER1 | QUEUE   | WARN    | Queue full (10/10). New task request
rejected.
[9.00] WORKER1 | PROC    | FAIL    | KV[0122] REJECTED - queue overflow. Notifying
Master.

[50.00] WORKER1 | PROC    | INFO    | Cumulative: processed=128, success=103,
fail=25
[100.00] WORKER1 | PROC    | INFO    | Cumulative: processed=513, success=411,
fail=102

[195.40] WORKER1 | STAT    | INFO    | === WORKER1 FINAL STATISTICS ===
[195.40] WORKER1 | STAT    | INFO    | Total tasks received      : 1,284
[195.40] WORKER1 | STAT    | INFO    | SUCCESS                   : 1,023  (79.7%)
[195.40] WORKER1 | STAT    | INFO    | FAIL                      : 261   (20.3%)
[195.40] WORKER1 | STAT    | INFO    | Avg waiting time         : 1.83 sec
[195.40] WORKER1 | STAT    | INFO    | P2P tasks transferred    : 38  (sent) / 21
(recv)
[195.40] WORKER1 | STAT    | INFO    | Total execution time      : 195.40 sec
[195.45] WORKER1 | TERMINATE | SUCCESS | Worker1 gracefully disconnected from
Master.
=====

```

4-3. Worker2.txt 예시 (실패 재할당 수신 케이스)

```

=== Worker2.txt (Worker Node 2 Log) ===

=====
Worker2 | Thread-based Worker | Ready Queue max=10
=====

[0.52] WORKER2 | CONNECT | SUCCESS | Connected to Master. Ready Queue initialized
(0/10).

[1.02] WORKER2 | RECV    | INFO    | Received task: KV[0003] (Key=c2b5, Value=55)
[1.02] WORKER2 | PROC    | INFO    | Processing KV[0003]... estimated time=1.95s
[2.97] WORKER2 | PROC    | FAIL    | KV[0003] FAILED (20% rule). Notifying Master.
[2.98] WORKER2 | PROC    | INFO    | FAIL message sent to Master for KV[0003].

[4.50] WORKER2 | RECV    | INFO    | Received PRIORITY task (reassigned): KV[0003-
retry]
[4.50] WORKER2 | PROC    | INFO    | Processing retry KV[0003]... (priority queue)
[6.44] WORKER2 | PROC    | SUCCESS | KV[0003] stored on retry. time=1.94s

[7.00] WORKER2 | RECV    | INFO    | Received P2P task from Worker1: KV[0088]
[7.00] WORKER2 | LB      | SUCCESS | ACK sent to Worker1. KV[0088] added to queue.
[9.10] WORKER2 | PROC    | SUCCESS | KV[0088] stored. time=2.10s

[20.00] WORKER2 | LB      | INFO    | Load check: queue=7/10. Broadcasting to
neighbors.
[20.01] WORKER2 | LB      | INFO    | Worker4 queue=3/10. Transferring 2 tasks to
Worker4.
[20.03] WORKER2 | LB      | SUCCESS | 2 tasks transferred. Queue: 7/10 -> 5/10

```

```

[195.40] WORKER2 | STAT | INFO | === WORKER2 FINAL STATISTICS ===
[195.40] WORKER2 | STAT | INFO | Total tasks received : 1,198
[195.40] WORKER2 | STAT | INFO | SUCCESS : 954 (79.6%)
[195.40] WORKER2 | STAT | INFO | FAIL : 244 (20.4%)
[195.40] WORKER2 | STAT | INFO | Retry tasks received : 187
[195.40] WORKER2 | STAT | INFO | Avg waiting time : 2.01 sec
[195.40] WORKER2 | STAT | INFO | P2P tasks transferred : 29 (sent) / 41
(recv)
[195.40] WORKER2 | STAT | INFO | Total execution time : 195.40 sec
[195.45] WORKER2 | TERMINATE | SUCCESS | Worker2 gracefully disconnected.
=====

```

4-4. Worker3.txt 예시 (Queue 초과 + 집중 수신 케이스)

=== Worker3.txt (Worker Node 3 Log) ===

```

=====
Worker3 | Thread-based Worker | Ready Queue max=10
=====

```

```

[0.53] WORKER3 | CONNECT | SUCCESS | Connected to Master. Ready Queue initialized
(0/10).

[1.00] WORKER3 | RECV | INFO | Received task: KV[0001] (Key=f9a3, Value=47)
[1.04] WORKER3 | RECV | INFO | Received task: KV[0005] (Key=e7d1, Value=12)
[3.20] WORKER3 | PROC | SUCCESS | KV[0001] stored. time=2.20s
[4.05] WORKER3 | PROC | SUCCESS | KV[0005] stored. time=2.01s (queue relief: 2
tasks remain)

[8.02] WORKER3 | LB | SUCCESS | Received 3 tasks from Worker1 (P2P): KV[0088-
90]
[8.02] WORKER3 | LB | INFO | Queue updated: 3/10 -> 6/10. ACK sent to
Worker1.

[10.50] WORKER3 | QUEUE | WARN | Queue full (10/10). New task request rejected
by Master rule.
[10.50] WORKER3 | RECV | FAIL | KV[0201] rejected - queue overflow (10/10).
Master notified.

[11.00] WORKER3 | PROC | SUCCESS | KV[0088] stored. time=1.88s. Queue: 9/10
[11.20] WORKER3 | PROC | FAIL | KV[0089] FAILED (20% rule). Notifying Master.
[12.10] WORKER3 | PROC | SUCCESS | KV[0090] stored. time=1.90s. Queue: 7/10

[50.00] WORKER3 | PROC | INFO | Cumulative: processed=138, success=110,
fail=28

[195.40] WORKER3 | STAT | INFO | === WORKER3 FINAL STATISTICS ===
[195.40] WORKER3 | STAT | INFO | Total tasks received : 1,312
[195.40] WORKER3 | STAT | INFO | SUCCESS : 1,048 (79.9%)
[195.40] WORKER3 | STAT | INFO | FAIL : 264 (20.1%)
[195.40] WORKER3 | STAT | INFO | Queue overflow rejects : 17
[195.40] WORKER3 | STAT | INFO | Avg waiting time : 1.97 sec
[195.40] WORKER3 | STAT | INFO | P2P tasks transferred : 14 (sent) / 52
(recv)
[195.40] WORKER3 | STAT | INFO | Total execution time : 195.40 sec
[195.45] WORKER3 | TERMINATE | SUCCESS | Worker3 gracefully disconnected.
=====

```

4-5. Worker4.txt 예시 (재할당 집중 처리 케이스)

```
=== Worker4.txt (Worker Node 4 Log) ===

=====
Worker4 | Thread-based Worker | Ready Queue max=10
=====

[0.54] WORKER4 | CONNECT | SUCCESS | Connected to Master. Ready Queue initialized (0/10).

[1.03] WORKER4 | RECV | INFO | Received task: KV[0004] (Key=h2k7, Value=91)
[2.80] WORKER4 | PROC | SUCCESS | KV[0004] stored. time=1.77s

[4.12] WORKER4 | RECV | INFO | Received PRIORITY task (reassigned): KV[0003-retry]
[4.12] WORKER4 | PROC | INFO | Processing priority task KV[0003-retry]...
[6.30] WORKER4 | PROC | SUCCESS | KV[0003-retry] stored. time=2.18s

[20.02] WORKER4 | LB | SUCCESS | Received 2 tasks from Worker2 (P2P):
KV[0340], KV[0341]
[20.02] WORKER4 | LB | INFO | Queue: 3/10 -> 5/10. ACK sent.
[22.10] WORKER4 | PROC | SUCCESS | KV[0340] stored. time=1.90s
[22.80] WORKER4 | PROC | FAIL | KV[0341] FAILED. Notifying Master.

[60.00] WORKER4 | PROC | INFO | Cumulative: processed=310, success=247,
fail=63

[195.40] WORKER4 | STAT | INFO | === WORKER4 FINAL STATISTICS ===
[195.40] WORKER4 | STAT | INFO | Total tasks received : 1,206
[195.40] WORKER4 | STAT | INFO | SUCCESS : 962 (79.8%)
[195.40] WORKER4 | STAT | INFO | FAIL : 244 (20.2%)
[195.40] WORKER4 | STAT | INFO | Priority (retry) tasks : 203
[195.40] WORKER4 | STAT | INFO | Avg waiting time : 1.91 sec
[195.40] WORKER4 | STAT | INFO | P2P tasks transferred : 18 (sent) / 37 (recv)
[195.40] WORKER4 | STAT | INFO | Total execution time : 195.40 sec
[195.45] WORKER4 | TERMINATE | SUCCESS | Worker4 gracefully disconnected.
=====
```

5. 과제 제출물

G조이름HW1.zip (ex. G1HW1.zip) // 압축 파일 오류 시 0점 처리

- 전체 소스 코드
- AllDefinedLogs.txt - 프로그램에서 정의한 모든 로그 메시지 명세 및 설명
- 결과 로그 파일: Master.txt, Worker1.txt, Worker2.txt, Worker3.txt, Worker4.txt - 각 노드에서 출력된 전체 로그
- download.txt - 5분 이내 시연 동영상(G 조이름 HW1.mp4) 다운로드 가능한 링크 포함 (ex. G1HW1.mp4)
 - 링크 오류, 공유 권한 없음, 재생 오류 등은 해당 조 과실로 감점
 - 영상은 전체 5,000 건 처리가 끝날 때까지 보여줄 필요는 없으며, 초기화·동적 분배·장애 재할당·P2P 부하분산·로그 출력 등 핵심 동작 과정이 확인되면 된다.
- Readme.txt - 아래 항목 포함:

- 조원 이름·학번·역할 명시
- 프로그램 구성요소 설명
- 소스코드 컴파일 및 실행 방법 (컴파일·실행 불가 시 과제전체 0 점 처리)
- 프로그램 실행 환경 및 실행 방법
- 동적 작업 분배 알고리즘 설명 (장단점 분석 포함) - 알고리즘 명시 필수
- P2P 부하 분산 알고리즘 설명 (장단점 분석 포함) - 알고리즘 명시 필수
- 장애 처리(Fault Tolerance) 메커니즘 설명
- 추가 구현 사항 및 기타 언급할 내용